10 CAHIERS DES CHARGES

Parcours progressif de développement web | du premier programme au projet full-stack

Public cible : étudiant en première année d'informatique. Objectif : réaliser 10 projets pédagogiques, déployer chaque réalisation gratuitement et construire un portfolio GitHub vérifiable.

Édition du 23 octobre 2026 | Périmètre : démonstration personnelle, sans usage commercial ni paiement réel.

Principe de progression

- Projets 1 à 6 : HTML, CSS, JavaScript, sans serveur à gérer.
- Projets 7 à 10 : application front-end et services Supabase ; l'étudiant apprend à manipuler des données persistantes et à sécuriser les accès.
- Chaque projet exige : dépôt GitHub public (sans secrets), README, captures d'écran, démonstration en ligne et tests documentés.
- Les durées estimées supposent que les notions sont apprises en parallèle ; la qualité prime sur le calendrier.

Répartition conseillée des hébergements

- GitHub Pages : pages statiques HTML/CSS/JavaScript (projets 1 à 6).
- Vercel Hobby : interfaces statiques et React de démonstration personnelle (projets 1 à 10). Attention : Hobby est réservé aux usages personnels non commerciaux.
- Supabase Free : authentification et PostgreSQL pour les projets 7 à 10. Le projet gratuit peut être suspendu après une période de faible activité ; le nombre de projets gratuits actifs est limité.
- Firebase Spark : alternative gratuite pour des prototypes front-end avec Auth/Firestore/Hosting sous quotas, mais ce dossier garde Supabase comme fil conducteur.
- Ne pas supposer que GitHub Pages héberge directement un backend Python, PHP ou Laravel : il sert les fichiers statiques.

Barème unique, sur 20 points

- Fonctionnalités obligatoires : 8 points.
- Fiabilité, validation et tests : 4 points.
- Qualité et lisibilité du code : 3 points.
- Design responsive et ergonomie : 2 points.
- README, GitHub et déploiement fonctionnel : 3 points.

Condition de passage au projet suivant

Au moins 14/20, toutes les exigences critiques vérifiées, et un lien de démonstration ouvrable sur un autre appareil. Les bonus ne compensent pas une faille de sécurité ou une fonction obligatoire absente.

PROJET 07 — Gestion des notes des étudiants

Niveau	7 / Premières données persistantes	Durée indicative	1 à 2 semaines
Technologies	HTML + CSS + JavaScript + Supabase (Postgres)	Déploiement	Vercel (interface) + Supabase (base)

1. Objectif du projet

Permettre à un enseignant fictif d'enregistrer des étudiants, matières et notes, puis de calculer des moyennes fiables.

2. Utilisateur cible et contexte

Enseignant de démonstration ; aucun nom réel d'étudiant nécessaire.

3. Fonctionnalités obligatoires

- Ajouter, modifier et retirer un étudiant.
- Créer des matières avec coefficient entier positif.
- Saisir une note de 0 à 20 pour un étudiant et une matière.
- Afficher le relevé de notes et la moyenne pondérée.
- Lister les étudiants, rechercher par nom et trier par moyenne.
- Ajouter un compte enseignant via Supabase Auth et restreindre l'accès aux données.
- Afficher les erreurs de validation et les états de chargement.

4. Pages / écrans attendus

- Connexion ; Liste des étudiants ; Fiche étudiant ; Matières ; Saisie des notes.

5. Données et structure

Tables : students(id, owner_id, name, student_code), subjects(id, owner_id, name, coefficient), grades(id, student_id, subject_id, value). Relations et contrainte d'unicité étudiant/matière.

6. Règles métier et contraintes

- La moyenne = somme(note × coefficient) / somme(coefficients des matières notées).
- Une matière non notée ne doit pas être assimilée silencieusement à zéro.
- Activer Row Level Security (RLS) : chaque enseignant voit uniquement ses données.
- Ne jamais mettre de clé service_role Supabase dans le navigateur.

7. Plan de réalisation conseillé

- Étape 1 : Créer la maquette et le schéma SQL.
- Étape 2 : Configurer Supabase Auth, relations et politiques RLS.
- Étape 3 : Construire les formulaires CRUD et le calcul des moyennes.
- Étape 4 : Ajouter listes, recherche et états d'erreur.
- Étape 5 : Déployer sur Vercel et tester l'isolation entre deux comptes.

8. Recette : tests d'acceptation

- Notes 10 (coef. 1) et 16 (coef. 2) donnent moyenne 14.

- Notes < 0 ou > 20 refusées.
- Utilisateur B ne voit ni ne modifie les dossiers de A.

9. Bonus facultatifs

- Exporter un relevé imprimable en PDF depuis le navigateur.
- Ajouter groupes/classes.

10. Livrables et critères de validation

- Dépôt GitHub nommé clairement, avec commits significatifs et fichier README (objectif, installation, utilisation, captures, limites).
- Lien HTTPS du site réellement accessible ; aucune clé secrète publiée dans le dépôt.
- Minimum trois captures d’écran : interface principale, fonctionnalité clé, affichage mobile.
- Petite démonstration de 2 à 5 minutes et checklist de tests signée par le mentor.